

CAFNT Project Launch Presentation

By: Lexa Hope, and, Heidi Lin.

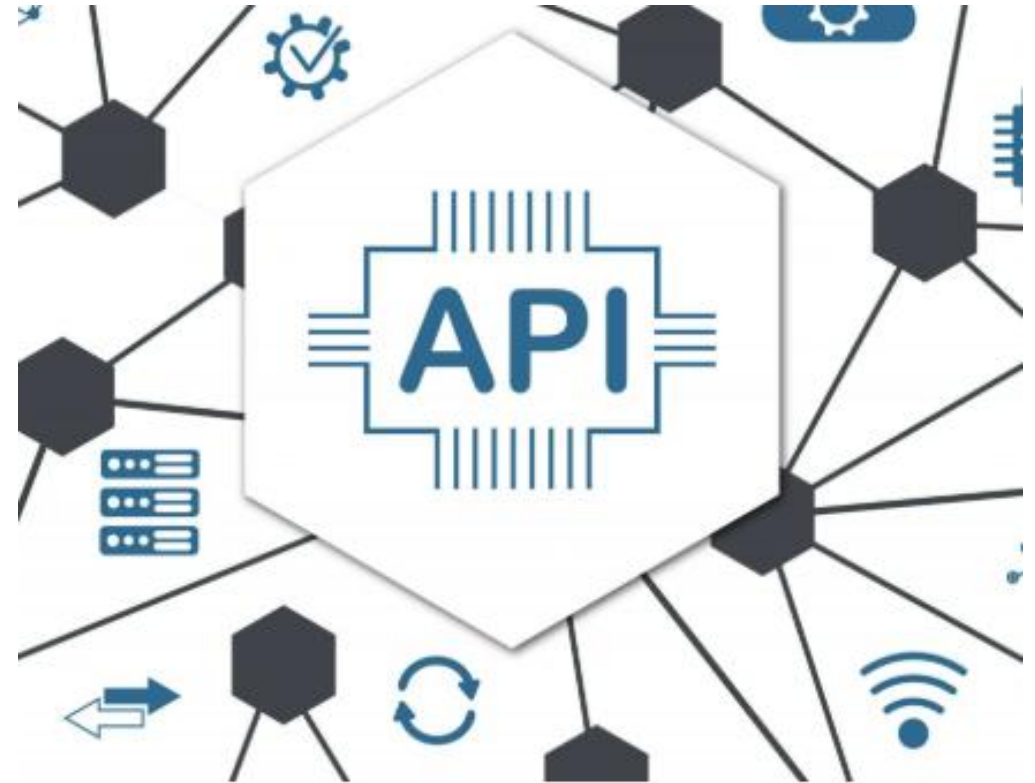


Brief Introduction

Our project focuses on creating a common API for communication using IR signals and serial interfaces. The main problem we are addressing is how to standardize communication between different input sources and hardware systems in a simple and flexible way.

How we approach

We designed a unified API that translates incoming signals, from an IR remote or a serial connection, into a consistent set of commands. These commands are then processed by the Arduino, which controls hardware outputs like LEDs and a buzzer.

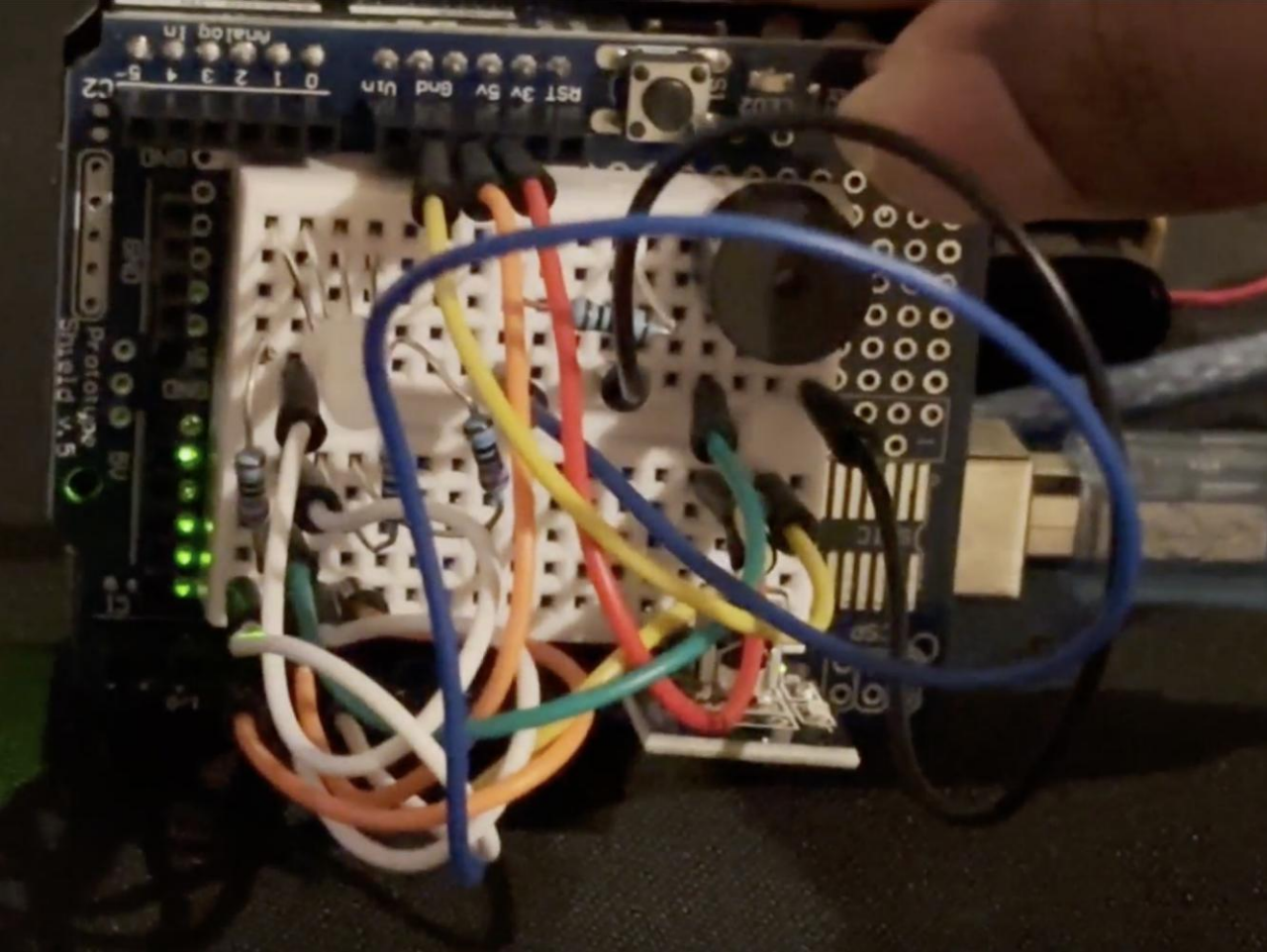


Inspire a belief and the rest will come:

We strive to provide the maximum services with our product. We have created different services for this product that allow for the system to be as compatible as required. This will allow for the product to work as required.

- Receiver.
 - CAFNT OS.
 - Serial Deamon.
 - Common API Extended (Coming Soon).
 - Recommended Board Layout for Receiver

Inspiring hope for the project using these guidelines related to the project.

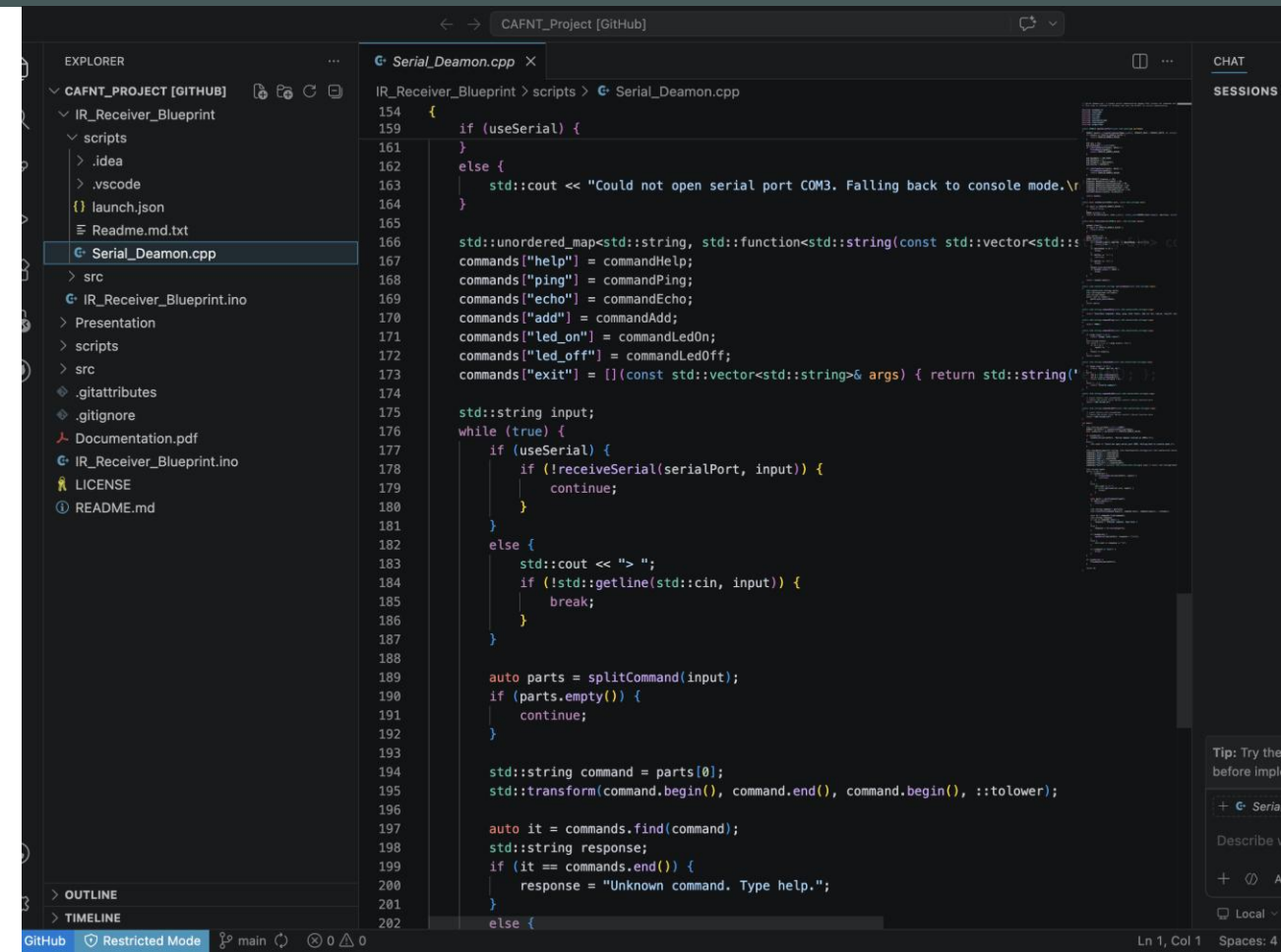


- Direct connection with mobile devices.
- Connection to IR broadcaster using remote.
- Low-profile devices with low power signature while in standby mode.
- Data parsed on device using CAFNT OS and then directly sent over serial connection to PC.
- Overloading function on PC permanent to added functionality to the protocol over time.

Simple Solution for Elegant Problem

Common API Extended

- Extended framework of build in framework in CAFNT OS.
- Runs directly on your PC or Mac.
- Allows for commands from the transmitter to be parsed by the receiver.
- Can be access through a GUI in the feature running on the local host as a webpage.



```
154 {
155     if (useSerial) {
161     }
162     else {
163         std::cout << "Could not open serial port COM3. Falling back to console mode.\r\n";
164     }
165
166     std::unordered_map<std::string, std::function<std::string(const std::vector<std::string>& args)>> commands;
167     commands["help"] = commandHelp;
168     commands["ping"] = commandPing;
169     commands["echo"] = commandEcho;
170     commands["add"] = commandAdd;
171     commands["led_on"] = commandLedOn;
172     commands["led_off"] = commandLedOff;
173     commands["exit"] = [] (const std::vector<std::string>& args) { return std::string(""); };
174
175     std::string input;
176     while (true) {
177         if (useSerial) {
178             if (!receiveSerial(serialPort, input)) {
179                 continue;
180             }
181         }
182         else {
183             std::cout << "> ";
184             if (!std::getline(std::cin, input)) {
185                 break;
186             }
187         }
188
189         auto parts = splitCommand(input);
190         if (parts.empty()) {
191             continue;
192         }
193
194         std::string command = parts[0];
195         std::transform(command.begin(), command.end(), command.begin(), ::tolower);
196
197         auto it = commands.find(command);
198         std::string response;
199         if (it == commands.end()) {
200             response = "Unknown command. Type help.";
201         }
202         else {
```

Any Questions:

